# Question 1: Understanding SRAMs

For this question, you may find it convenient to reference the `sram22_sky130_macros` repository, which stores all the SRAM variants that are directly supported in our tool flow.

a) How many SRAM sizes are available? Pick one size and describe what each number in the name of the macro means. The SRAM22 documentation may be helpful.
There are 22 different SRAM sizes are available in SRAM22.
Take the first one sram22_64*22m4w22 as an example, it has 64 entries deep and each entry is 22-bits wide. Mux size is 4 and write size is 22.




b) Look at one of the SRAM Verilog model (`*.v`) files. Based on the model, what happens to the `dout` port when a write operation is performed? Note that other SRAM designs may have different behavior.
For sram22_64*22m4w22, nothing happen to dout when a write operation is performed

c) Open one of the SRAM abstract ( `*.lef` ) files. Where are the pins located? Which layer are they on? What layer are the power straps on?
For sram22_64*22m4w22, the pins are located on layer 1 and 2.
There are 22 dout pins, 22 dout pins, 1 wmask pin, 6 address pins and some other pins.
For example Pin dout[0] is located from [139.15, 0] to [139.29, 0.14]
All the pins are located on the bottom edge of the die.
Located the command for pin vdd and vss, the power straps are on layer 2.

For determining the pin location, use the 2D coordinates to determine the location of the pin in relation to the origin at the top of the file.

d) (Ungraded thought experiment #1) SRAM libraries in real process technologies are much larger than the list you see in the build directory. What features do you think are important for real SRAM libraries? Think in terms of number of ports, masking, improving yield, or anything else you can think of. What would these features do to the size of the SRAM macros?
I think the following features are important for real SRAM libraries:
1. Number of Input/output
2. Yield of the SRAM
3. Power consumption
4. Redundancy

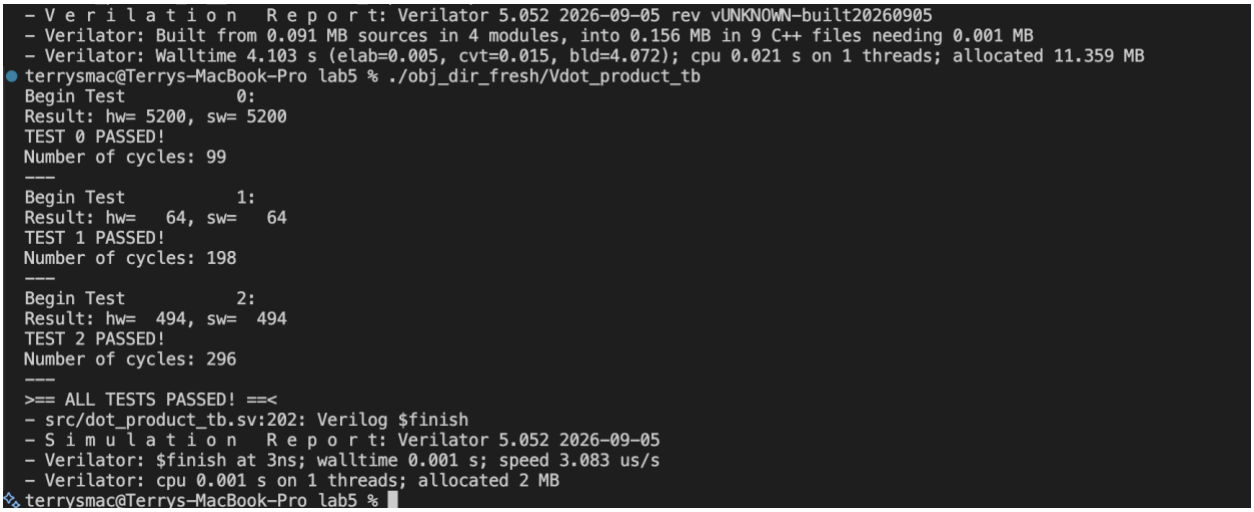
e) (Ungraded thought experiment #2) SRAMs should be integrated very densely in a circuit's layout. To build large SRAM arrays, often times many SRAM macros are tiled together, abutted on one or more sides. Knowing this, take a guess at how SRAMs are laid out.
we can stack the SRAM, since each SRAM uses only 2 layers. Also, SRAM can be placed next to each other SRAM tightly.

i) In Sky130, there are only 5 metal layers, but realistically only 4 layers to route on in order to leave the top layer for power distribution, as you saw in Lab 4.* *How many layers should a well-designed SRAM macro use (i.e. block off from PAR routing), at maximum?
At least 2 layer. From the lef file, SRAM use at least 2 layer

ii) Where should the pins on SRAMs be located, if you want to maximize the ability for them to abut together?
Pins of the SRAMS should be placed side by side, and I think it is better for them to place on the same edge

# Question 2: Dot Product Design

a) Upload a screenshot of your terminal output showing that your design passes the **post-syn** tests.




b) Provide the SystemVerilog assertions that you included in your design.

```
//Assertion 1: check that only one input can have handskae
Single_Handshake: assert property (
  @(posedge clk) disable iff (rst)
  !(a_fire && b_fire)
) else $error("Both A and B accepted in same cycle!");

//Assertion 2: output signal should stay stable
Stable_Output: assert property (
  @(posedge clk) disable iff (rst)
  (c_valid && !c_ready) |=> (c_valid && $stable(c_data))) else $error("Output handshake signal is not stable!");
```

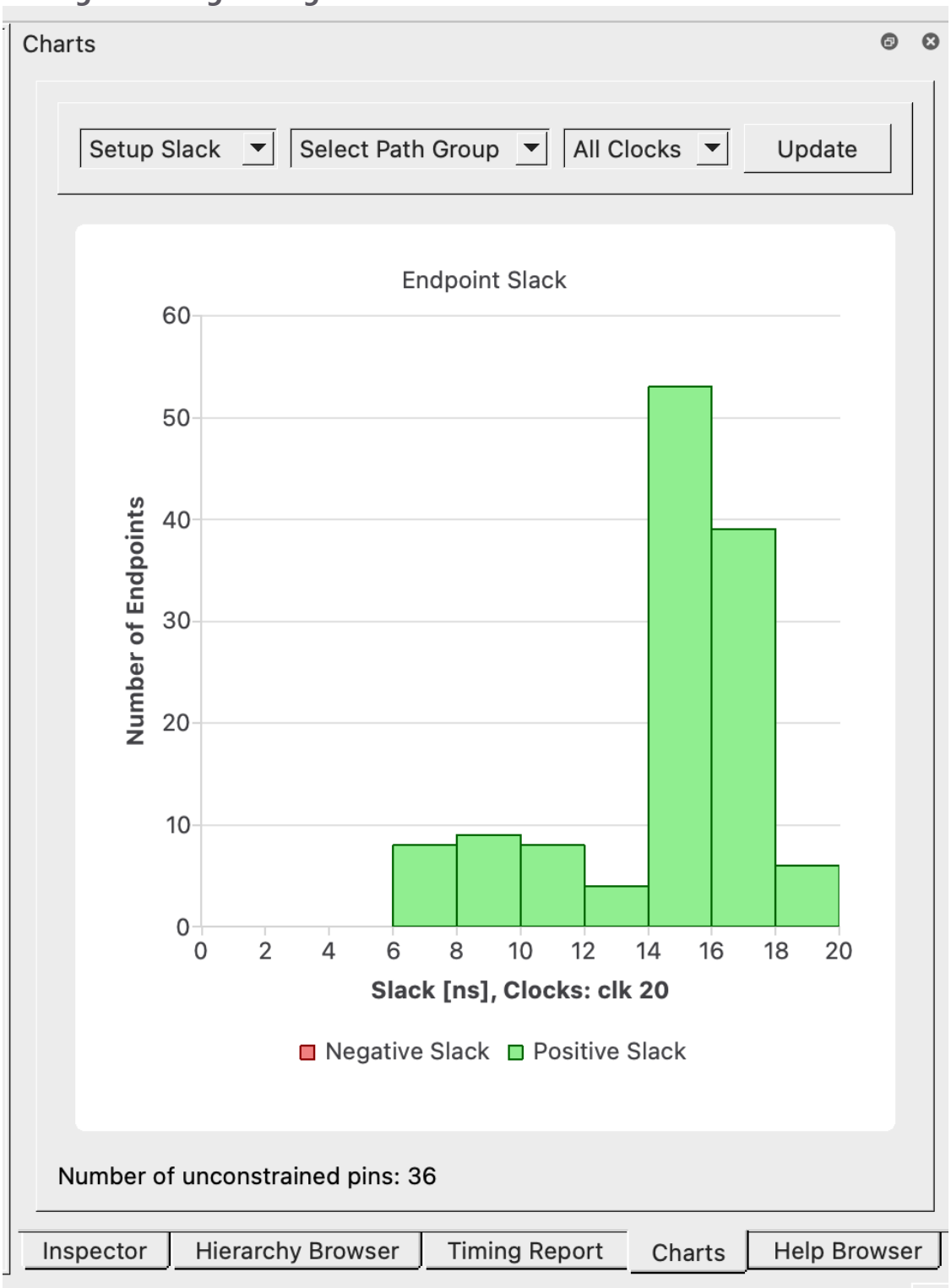
# Question 3: Timing

a.) Check the post-Synthesis timing report ( `syn-rundir/reports/final_time_ss_100C_1v60.setup_view.rpt` ) and post-PAR timing report ( `par-rundir/timingReports/dot_product_postRoute_all.tarpt` ). **For each report, identify the critical path (startpoint and endpoint). Do the post-synthesis and post-PAR critical paths correspond to the same logical path?**
The critical path is from SRAM to flipflop_5803.
Same

b.) Iterate on your design by adjusting the clock period in `design.yml` to obtain a rough estimate of the minimum achievable period. Don't forget to run synthesis ( `make sim-gl-syn` ) in addition to par for each iteration!

c.) Open the post-CTS timing report( `par-rundir/timingReports/dot_product_postCTS_all.tarpt` ) and the post-PAR timing report( `par-rundir/timingReports/dot_product_postRoute_all.tarpt` ). **Find a common path (same start and end points). What differences do you notice within the paths?**
Same critical path from SRAM to flipflop_5803.
Slack is less in post-cts timing report

d.) Submit a screenshot of your setup slack path histogram. In the Innovus GUI, go to **Timing** –> **Debug Timing** –> **OK**.



# Question 4: Performance and Area Optimization

a) Submit a screenshot of the output from running `make sim-gl-par` showing that you pass all tests using your post-PAR design at your minimum clock period. Note that you'll need to update the `CLOCK_PERIOD` in `sim-gl-par.yml` to match the frequency at which you ran synthesis/PAR.

```
85   ===== 7. Timing after optimization =====
86   worst slack max 7.404
87   worst slack min 0.364
88   tns max 0.000
89   ===== 8. Save optimized checkpoint =====
90   Saved: /Users/terrysmac/Documents/eecs151-asic-lab-sp26/lab5/local_openroad/results/dot_product_cts_opt
91   ===== Post-CTS optimization completed =====
92
```

b) The floorplan we've given you has lots of empty space. Adjust the SRAM position and design area bounds to reduce the overall area of your floorplan by a factor of 1/2. Make sure that your design is still passing timing. Report the final dimensions of your floorplan, and submit a screenshot of your post-PAR layout. You must not have any DRC errors (i.e. no white X's on your layout).
x_max: 600 um
y_max: 500 um
70% reduction on die area

c) How many cycles does your dot product module take for each of the test cases? Describe two ways to reduce the total cycle count. You don't need to implement these changes. You are free to make reasonable modifications to the structure of the problem (e.g. you can change the inputs to be something other than ready/valid interfaces).
test 0 is all A before B. it runs 99 cycles
test 1 is all B before A. it runs 99 cycles
test 2 is interleave both A and B. it runs 98 cycles
It is better to have a dual port to read A and B separately. From the Routing congestion result on GUI, the congestion mainly happens at the output of SRAM

d) Open your final design’s floorplan. Identify one or more locations where the SRAM is connected to power and ground, and submit a screenshot in your report.

